

SCR2043 OPERATING SYSTEMS

Name : LAU YEE WEN
Student ID : A23CS0099
Section : 02

Marks

This lab assessment is designed to test your understanding and skills on some basic concepts and tools related to process monitoring and management in operating system. Please follow the instructions carefully and submit your answers in this word document and rename the file as **os-lab-assessment02-studentname-matricno.docx**.

Essential Steps Before Starting Lab Assessment 2:

1. Download necessary source codes:

Use the `wget` command to retrieve the following source code files to your Linux (or WSL or MacOS) environment:

```
wget -O mainprocess.c https://rebrand.ly/mainprocess_c  
wget -O subprocess1.c https://rebrand.ly/subprocess1_c  
wget -O subprocess2.c https://rebrand.ly/subprocess2_c
```

2. Compile the source files:

Use the `gcc` compiler to create executable files from the source code.

```
gcc mainprocess.c -o mainprocess  
gcc subprocess1.c -o subprocess1  
gcc subprocess2.c -o subprocess2
```

3. Execute the dummy processes:

Run all the dummy processes

```
./mainprocess &
```

Press **enter** two times.

4. The dummy processes are running for 2 hours. If you took longer than 2 hours on questions 1-9, please restart the main process with `./mainprocess &`.

Lab Assessment 2 : Linux Process Monitoring and Management

Instructions:

1. Carefully execute each command as instructed in the questions.
2. Write down the exact command used for each task.
3. Capture a screenshot of the command's output.

Question 1

Use the `ps` command with the appropriate option to display a complete list of all running processes within the Linux operating system.

Command			
ps -e			
Output			
917	pts/0	00:00:00	mainprocess
918	pts/0	00:00:00	mainprocess
919	pts/0	00:00:00	mainprocess
920	pts/0	00:00:00	subprocess2
921	pts/0	00:00:00	subprocess2
922	pts/0	00:00:00	subprocess1
923	pts/0	00:00:00	subprocess2
924	pts/0	00:00:00	subprocess1
925	pts/0	00:00:00	ps

Question 2

Employ the `ps` command with necessary options to unveil comprehensive details about each running process.

Command			
ps -ef grep -E 'mainprocess subprocess'			
Output			
<pre>yeewen@secre2043:~\$ ps -ef grep -E 'mainprocess subprocess' yeewen 917 873 0 13:05 pts/0 00:00:00 ./mainprocess yeewen 918 917 0 13:05 pts/0 00:00:00 ./mainprocess yeewen 919 917 0 13:05 pts/0 00:00:00 ./mainprocess yeewen 920 919 0 13:05 pts/0 00:00:00 ./subprocess2 yeewen 921 919 0 13:05 pts/0 00:00:00 ./subprocess2 yeewen 922 918 0 13:05 pts/0 00:00:00 ./subprocess1 yeewen 923 919 0 13:05 pts/0 00:00:00 ./subprocess2 yeewen 924 918 0 13:05 pts/0 00:00:00 ./subprocess1 yeewen 931 873 33 13:06 pts/0 00:00:00 grep --color=auto -E mainprocess subprocess yeewen@secre2043:~\$</pre>			

Question 3

Use the `ps` command with some tools to only list processes named "subprocess" and show some info about them.

Command	
<code>ps -ef grep -E 'subprocess'</code>	
Output	
<pre>yeewen@secr2043:~\$ ps -ef grep -E 'subprocess' yeewen 920 919 0 13:05 pts/0 00:00:00 ./subprocess2 yeewen 921 919 0 13:05 pts/0 00:00:00 ./subprocess2 yeewen 922 918 0 13:05 pts/0 00:00:00 ./subprocess1 yeewen 923 919 0 13:05 pts/0 00:00:00 ./subprocess2 yeewen 924 918 0 13:05 pts/0 00:00:00 ./subprocess1 yeewen 935 873 0 13:07 pts/0 00:00:00 grep --color=auto -E subprocess yeewen@secr2043:~\$ </pre>	

Question 4

Execute the `ps` command, specifying options that reveal only the following columns:

- Process ID (`pid`)
- Owner of the process (`user`)
- CPU percentage (`pcpu`)
- Memory percentage (`pmem`)
- Command (`cmd`)

Command	
<code>ps -eo pid,user,pcpu,pmem,cmd grep -E 'subprocess'</code>	
Output	
<pre>yeewen@secr2043:~\$ ps -eo pid,user,pcpu,pmem,cmd grep -E 'subprocess' 920 yeewen 0.0 0.1 ./subprocess2 921 yeewen 0.0 0.1 ./subprocess2 922 yeewen 0.0 0.1 ./subprocess1 923 yeewen 0.0 0.1 ./subprocess2 924 yeewen 0.0 0.1 ./subprocess1 937 yeewen 50.0 0.1 grep --color=auto -E subprocess yeewen@secr2043:~\$</pre>	

Question 5

Building on the `ps` command used in Question 4, can you add an option to sort the listed processes by their memory usage (`pmem`)?

Command
<code>ps -eo pid,user,pcpu,pmem,cmd --sort=pmem grep -E 'subprocess'</code>
Output
<pre>yeewen@secr2043:~\$ ps -eo pid,user,pcpu,pmem,cmd --sort=pmem grep -E 'subprocess' 920 yeewen 0.0 0.1 ./subprocess2 921 yeewen 0.0 0.1 ./subprocess2 922 yeewen 0.0 0.1 ./subprocess1 923 yeewen 0.0 0.1 ./subprocess2 924 yeewen 0.0 0.1 ./subprocess1 941 yeewen 100 0.1 grep --color=auto -E subprocess yeewen@secr2043:~\$</pre>

Question 6

Construct a command using `ps`, suitable options, and any additional tools to visualize the hierarchical structure (tree-like) of the following processes:

- "mainprocess"
- "subprocess1"
- "subprocess2"

Command
<code>ps --forest -C mainprocess -C subprocess1 -C subprocess2</code>
Output
<pre>yeewen@secr2043:~\$ ps --forest -C mainprocess -C subprocess1 -C subprocess2 PID TTY TIME CMD 917 pts/0 00:00:00 mainprocess 918 pts/0 00:00:00 _ mainprocess 922 pts/0 00:00:00 _ subprocess1 924 pts/0 00:00:00 _ subprocess1 919 pts/0 00:00:00 _ mainprocess 920 pts/0 00:00:00 _ subprocess2 921 pts/0 00:00:00 _ subprocess2 923 pts/0 00:00:00 _ subprocess2 yeewen@secr2043:~\$</pre>

Question 7

Use `pstree` command with option that show the number of threads to each process.

Command
<pre>pstree -c -s 917</pre>
Output
<pre>yeewen@secr2043:~\$ pstree -c -s 917 systemd---sshd---sshd---sshd---bash---mainprocess+-mainprocess+-subprocess1 -subprocess1 +-mainprocess+-subprocess2 -subprocess2 +-subprocess2</pre>

Question 8

Use renice command to change priority level of one of process “subprocess1”.

Command
<pre>sudo renice -5 922</pre>
Output
<pre>yeewen@secr2043:~\$ ps -o pid,nice,comm -C subprocess1 PID NI COMMAND 922 0 subprocess1 924 0 subprocess1 yeewen@secr2043:~\$ sudo renice -5 922 [sudo] password for yeewen: 922 (process ID) old priority 0, new priority -5</pre>

Question 9

Terminate all running processes with the name “mainprocess”.

Command
killall -15 mainprocess
Output
<pre>yeewen@secr2043:~\$ killall -15 mainprocess Main process (ID: 917) received signal: 15. Terminating... Main process (ID: 919) received signal: 15. Terminating... Main process (ID: 918) received signal: 15. Terminating... yeewen@secr2043:~\$ killall -15 mainprocess mainprocess: no process found [1]+ Done ./mainprocess</pre>

Question 10

Write a short C or Python code (choose only one language) demonstrating multiprocessing with `fork()` and `wait()`. Compile and/or run the code. Show the output.

Source Code:

```
nano example.c
gcc example.c -o example
gcc ./example

example.c

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>

void child_process() {
printf("Child process with PID: %d\n", getpid());
int sleep_time = rand() % 5 + 1;
printf("Child process sleeping for %d seconds\n", sleep_time);
sleep(sleep_time);
printf("Child process exiting\n");
}

int main() {
printf("Parent process with PID: %d\n", getpid());
// Fork a child process
pid_t pid = fork();
if (pid == 0) {
// This is the child process
child_process();
exit(0);
} else if (pid > 0) {
// This is the parent process
printf("Parent process waiting for child process to finish\n");
// Wait for the child process to finish
wait(NULL);
printf("Parent process exiting\n");
} else {
// Error occurred while forking
perror("fork");
return 1;
}
return 0;
}
```

Output:

```
yeewen@secre2043:~$ nano example.c
yeewen@secre2043:~$ gcc example.c -o example
yeewen@secre2043:~$ ./example
Parent process with PID: 976
Parent process waiting for child process to finish
Child process with PID: 977
Child process sleeping for 4 seconds
Child process exiting
Parent process exiting
```

```
yeewen@secre2043: ~
GNU nano 7.2 example.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>

void child_process() {
    printf("Child process with PID: %d\n", getpid());
    int sleep_time = rand() % 5 + 1;
    printf("Child process sleeping for %d seconds\n", sleep_time);
    sleep(sleep_time);
    printf("Child process exiting\n");
}

int main() {
    printf("Parent process with PID: %d\n", getpid());
    // Fork a child process
    pid_t pid = fork();

    if (pid == 0) {
        // This is the child process
        child_process();
        exit(0);
    } else if (pid > 0) {
        // This is the parent process
        printf("Parent process waiting for child process to finish\n");
        // Wait for the child process to finish
        wait(NULL);
        printf("Parent process exiting\n");
    } else {
        // Error occurred while forking
        perror("fork");
        return 1;
    }

    return 0;
}
```